

DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 11

Stream Processing

Unbounded data, ordered logs, and keeping derived systems in sync without dual-write

Source: [claude-skills / ch11-stream-processing.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part III: Derived Data

Table of Contents

1. [Core Idea](#)
2. [Framework: Log-Based Kafka](#)
3. [Framework: AMQP/JMS Brokers](#)
4. [Partitioned Logs & Consumer Offsets](#)
5. [Log Compaction](#)
6. [Dual-Write Problem](#)
7. [Change Data Capture \(CDC\)](#)
8. [Event Sourcing & Commands vs Events](#)
9. [Event Time vs Processing Time](#)
10. [Window Types](#)
11. [Stream Joins](#)
12. [Exactly-Once Semantics](#)
13. [Mental Models](#)
14. [Anti-Patterns](#)
15. [Worked Example: CDC Search Index Sync](#)
16. [Problems Each Mechanism Solves](#)
17. [Connects To \(Cross-Chapter\)](#)
18. [Key Takeaways](#)
19. [Junior SWE Checkpoints](#)

1. Core Idea

Batch assumes bounded input; real data is **unbounded**. Stream processing is the incremental counterpart: process every event as it happens instead of chunking by day.

Batch

Bounded input, periodic jobs, reprocess whole dataset.

Stream

Unbounded input, incremental processing, low latency per event.

Same coin

Database writes *are* an event stream; expose the log and derived systems follow.

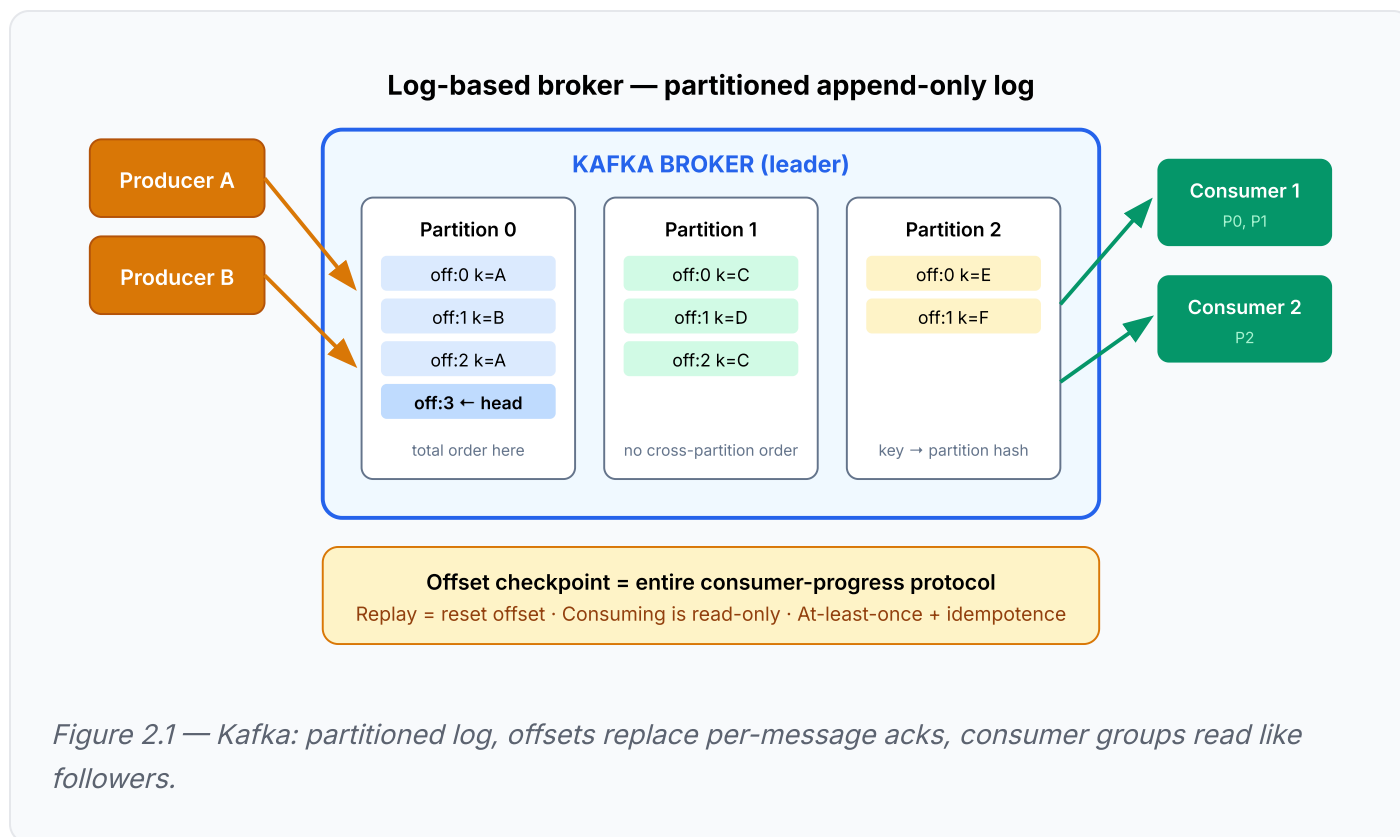
Databases and streams are two sides of the same coin. A replication log is already a stream of write events. Exposing it (CDC) or modeling the app on events (event sourcing) keeps caches, indexes, and warehouses consistent with one system of record.

"The truth is the log. The database is a cache of a subset of the log." — Pat Helland

2. Framework: Log-Based Kafka

Also: Amazon Kinesis Streams, Twitter DistributedLog, Kafka Streams, Samza. **Default for data integration** — high throughput, replay, derived state.

When	High throughput, ordering within a key, consumers need replay, or the stream feeds derived state (indexes, caches, other streams).
How	Producers append to a partitioned, append-only log on disk. Each partition assigns a monotonically increasing offset per message. Consumer groups are assigned whole partitions; each consumer reads sequentially, single-threaded, periodically checkpointing its offset.
Why it works	Per-partition offset replaces per-message acks — broker behaves like a leader database, consumer like a follower resuming from a log sequence number. Kills ack bookkeeping; enables batching/pipelining at millions of msgs/sec while writing everything to disk.
Failure mode	Parallelism capped at partition count; one slow message head-of-line-blocks its partition; consumer lagging past segment retention silently misses messages; dying after processing but before committing offset → reprocessing on restart.



Also named: Storm, Spark Streaming (microbatch), Flink (checkpointing), Google Cloud Dataflow, Debezium, Maxwell, Bottled Water, Kafka Connect, Esper (CEP), Event Store.

3. Framework: AMQP/JMS Brokers

Also: RabbitMQ, ActiveMQ, IBM MQ, Azure Service Bus. **Task queues** — async RPC, not data integration.

When	Task queues — expensive per-message work, order unimportant, no replay needed. Think async RPC.
How	Broker assigns individual messages. Consumer acks each; broker deletes on ack . Load-balances across competing consumers.
Why it works	Simple per-message handoff; no log retention overhead; good for fire-and-forget jobs.
Failure mode	Load balancing + redelivery reorders messages (crashed consumer's unacked m3 redelivered after m4). Consuming is destructive — no re-runs; new consumer sees no history.

Two broker families, two contracts

Log-based (Kafka)

Partitioned · Ordered · Replayable
Offset checkpoint · Fan-out free
Derived state · Data integration

Consumer = follower replaying log

AMQP/JMS (RabbitMQ)

Per-message ack · Destructive
Unordered under redelivery
Task queue · Async RPC

No history for new consumers

Figure 3.1 — Pick the broker family by whether you need replay and ordered derived state.

 **Kafka = library lending log** (check out, return, re-read any page). **AMQP = passing notes that burn after reading** — fast handoff, no archive.

4. Partitioned Logs & Consumer Offsets

Total order **within** a partition only — none across partitions. Same key → same partition → per-key ordering preserved.

Concept	Detail
Offset	Monotonically increasing position per partition. Replaces per-message ack bookkeeping.
Consumer group	Each partition assigned to exactly one consumer in the group. Parallelism = partition count.
Checkpointing	Consumer periodically commits offset = entire progress protocol. Replay = reset offset.
Head-of-line blocking	One slow message blocks its entire partition — can't skip ahead within a partition.
Retention lag	Consumer lagging past segment retention silently misses messages — monitor lag!

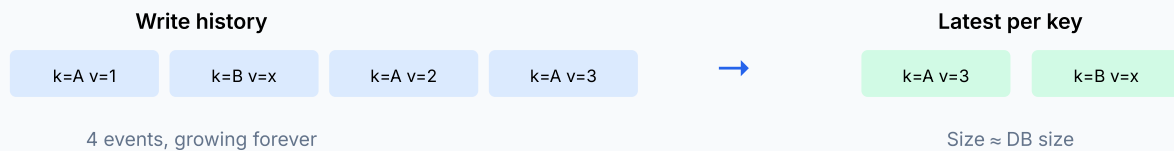
Crash between process and commit: event processed, offset not committed → reprocessing on restart.
Plan for **at-least-once + idempotence**, not naive exactly-once assumptions.

5. Log Compaction

Background process keeps only the **latest value per key** (null = tombstone/delete). Compacted log size tracks *database size*, not write history.

A topic can hold a **full copy of a database** forever. Kafka supports this natively — the load-bearing trick behind Kafka-as-storage and cheap rebuilds of derived views.

Before compaction → After compaction



New consumer: start offset 0, scan to head → caught up. No snapshot coordination.

Figure 5.1 — Compacted topic = durable full dataset for bootstrap without DB load.

6. Dual-Write Problem

App writes DB, then index, then cache — two failure modes guarantee silent divergence:

Race

Concurrent clients interleave: DB ends with B, index with A — permanently inconsistent, **no error raised**.

Partial failure

DB write succeeds, index write fails — systems desync with no rollback.

Root cause

No single leader ordering writes across systems.

The dual-write trap

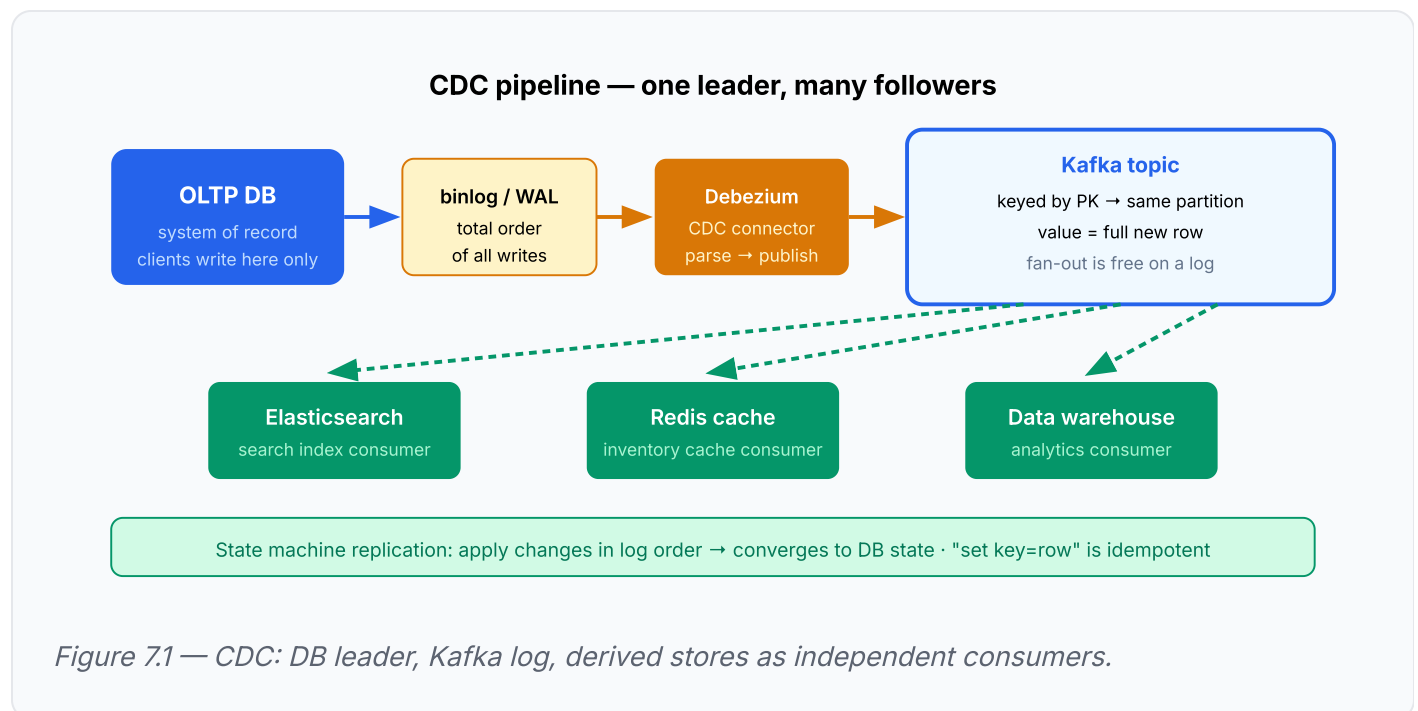
Request path: `UPDATE users SET name='Alice' → Elasticsearch.index(user) → Redis.set(user)`

Two concurrent updates to the same user can land in different orders in each store. You can't detect it, can't fix it retroactively. **Fix:** one system of record + CDC/event log; derived stores are followers.

7. Change Data Capture (CDC)

Parse the DB's replication log, publish changes as a stream: DB becomes **leader**, derived systems become **followers** applying the *same* order.

When	Keep cache, search index, warehouse, or analytics in sync without dual-write.
How	CDC connector (Debezium, Maxwell, Bottled Water) tails binlog/WAL → publishes keyed change events to Kafka. Consumers apply in log order.
Bootstrap	Raw log is truncated — normally need snapshot tied to log offset. Or: compacted topic with full row per event → no snapshot needed.
Caveat	Async — replication lag applies. User may not see own write in search (read-your-writes violation on derived store).



Real systems: Debezium (Postgres WAL, MySQL binlog), Maxwell, Bottled Water, Kafka Connect. Hotel reservation Redis cache, email service Elasticsearch reindex (Ch 5 logical log → Ch 11 CDC).

8. Event Sourcing & Commands vs Events

Event sourcing — same log idea at the application level: append-only log of immutable, intent-carrying domain events ("seat reserved"), state derived by deterministic replay.

Later events don't supersede earlier ones (unlike CDC row updates), so **compaction doesn't apply** — snapshot state as a performance optimization only.

Commands vs Events

Type	Properties	Handling
Command	Intent — may fail validation ("reserve seat 7A")	Validate synchronously <i>before</i> append
Event	Fact — immutable once accepted ("SeatReserved{7A, user42}")	Consumers cannot reject events. Split into tentative + confirmation if needed.

📁 **Accountant's ledger**: never erase — append a compensating event. Immutability buys auditability, recovery from buggy deploys, and new read views derived from history. **CQRS**: write shape $\neq$ read shape, so denormalizing read views is safe.

9. Event Time vs Processing Time

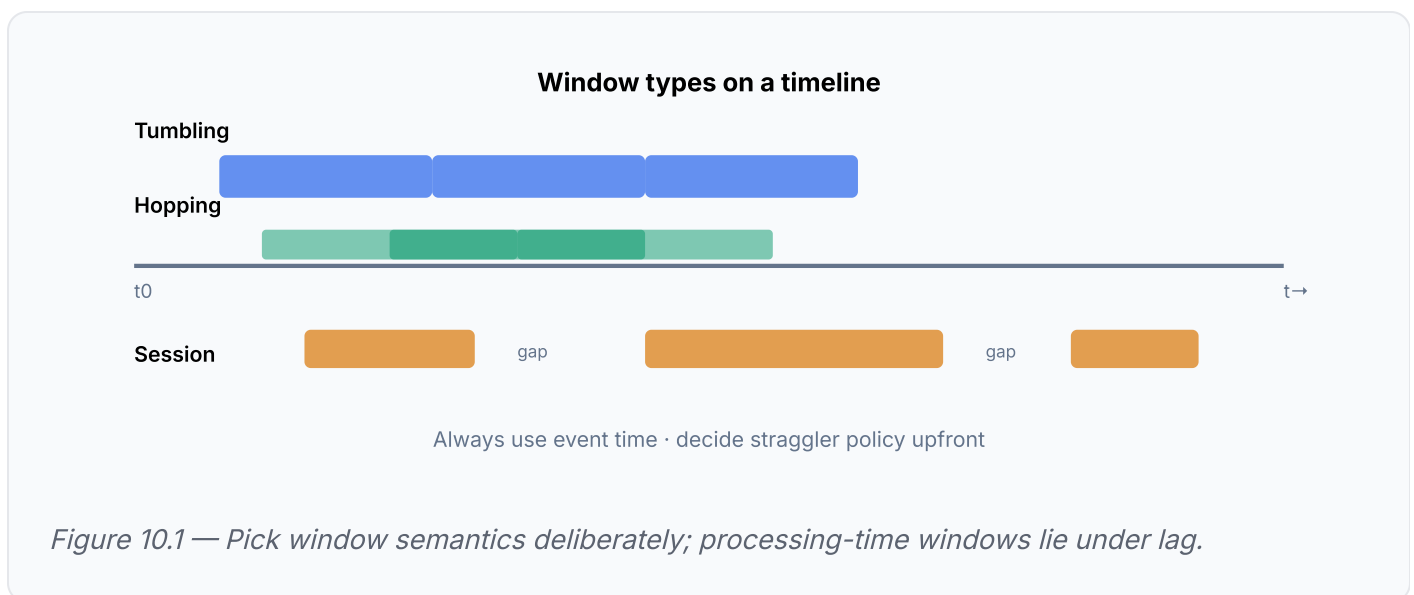
Windowing on the processor's clock creates artifacts — a restarted processor's backlog looks like a request spike. **Window on event timestamps.**

Timestamp	Source	Problem
Event time	When the event actually occurred (device clock)	Correct for analytics; windows may never be "complete"
Processing time	When the processor handles the event	Redeploys/lag manufacture fake traffic spikes
Ingestion time	When server received the event	Server clock more reliable than device

Stragglers: events arrive late — you can never be sure a window is complete. Either **drop them** (track how many) or **emit a correction**. Device clocks lie: log three timestamps (occurred, sent per device clock, received per server clock) to estimate offset.

10. Window Types

Type	Behavior	Use when
Tumbling	Fixed length, non-overlapping (e.g. 5-min buckets)	Periodic rollups — metrics per minute
Hopping	Fixed length, overlaps by hop size (e.g. 5-min window, 1-min hop)	Smoothed moving averages
Sliding	All events within an interval of each other — no fixed boundaries	"Events within 30s of each other"
Session	No fixed length; closes on inactivity gap (e.g. 30 min idle)	User sessions, click streams



11. Stream Joins

All three join types reduce to: **maintain state from one input, probe from the other**. Cross-stream ordering is undefined → joins are nondeterministic on replay.

Join type	Pattern	Example
Stream-stream	Window join: both sides indexed in local state within a time window	Search event + click event by session ID → click-through rate
Stream-table	Enrichment via CDC-fed local replica instead of remote DB per event	Click event joined with user profile from changelog-maintained table
Table-table	Join two changelogs to maintain a materialized view	Twitter timelines — tweets changelog × follows changelog

Don't embed search details in click events instead of joining — loses no-click searches; click-through rate becomes uncomputable. **Don't query remote DB per event** — slow, overloads DB; keep CDC-fed local replica.

Materialized join follows the product rule: $(u \cdot v)' = u'v + uv'$ — state is the integral of the change stream.

12. Exactly-Once Semantics

The **visible effect** is as-if-once even though retries reprocess. Not a magic delivery mode — built from at-least-once + careful output handling.

Layer	Mechanism	Limit
Inside framework	Microbatching (Spark Streaming), checkpointing with barriers (Flink)	Works within the processor
Output escapes	DB writes, emails, external APIs	Need atomic commit of output + state + offset, or idempotence
Idempotence pattern	Write Kafka offset alongside value in target DB; skip already-applied updates	Requires deterministic processing, identical replay order (log gives this), fencing against zombie nodes

- 1 **At-least-once delivery** — accept retries, commit offset after processing
- 2 **Idempotent writes** — "set key=row" or dedup by event ID/offset
- 3 **Atomic commit** — 2PC: output + state + offset in one transaction (Ch 7/9)
- 4 **Fencing** — prevent zombie processors from writing stale results (Ch 9)

🚩 **At-least-once redelivery without idempotence** — every retry double-counts. A few percent double-counting error is real financial loss in ad click aggregation.

13. Mental Models

1. Streams ↔ tables (calculus duals)

State is the **integral** of the change stream; the changelog is the **derivative** of state. Materialized join: $(u \cdot v)' = u'v + uv'$.

2. Broker as leader, consumer as follower

Log-based broker is database replication wearing a different hat. Offsets are log sequence numbers (Ch 5).

3. CEP inverts the database

Databases store data, queries are transient. CEP stores queries long-term, streams data past them (Esper).

4. The accountant's ledger

Never erase — append compensating events. Immutability → auditability, replay, new read views. CQRS: write shape ≠ read shape.

14. Anti-Patterns

Anti-pattern	Why it fails
Dual writes to keep systems in sync	Races and partial failures guarantee silent divergence. Use CDC / single ordered log.
Windowing by processing time when lag is possible	Redeploys manufacture fake traffic spikes.
At-least-once redelivery without idempotence	Every retry double-counts — real financial loss in ad aggregation.
Embedding search details in click events instead of joining	Loses no-click searches; click-through rate uncomputable.
Querying a remote DB per event for enrichment	Slow, overloads DB; keep CDC-fed local replica.
Assuming event log satisfies deletion regulations	Need excision-style history rewrite; truly deleting is hard (SSDs, backups).

15. Worked Example: CDC Search Index Sync

- 1 **OLTP database** is system of record; clients write only to it. Replication log totally orders every write.
- 2 **CDC connector** (Debezium) publishes each change — keyed by PK, value = full new row — to Kafka topic partitioned by key. Same key → same partition → per-key ordering preserved.
- 3 **Search index consumer** applies changes in log order → state converges to DB (state machine replication). Cache and warehouse are additional consumers — fan-out is free.
- 4 **Bootstrap new index:** enable log compaction — each event carries full latest row, compacted topic *is* a full DB copy. New consumer starts offset 0, scans to head, caught up. No snapshot coordination, no load on source DB.
- 5 **Fault tolerance:** commit offsets after applying writes. Crash → few events replay. "Set key=row" is naturally idempotent → exactly-once effect without transactions.
- 6 **Caveat:** CDC is async — replication lag applies. User may not see own write in search (read-your-writes violation on derived store).

User updates profile name in Postgres.

- binlog: {op: UPDATE, pk: 42, after: {name: "Alice", ...}}
- Debezium → Kafka topic "users" partition = hash(42)
- ES consumer: PUT /users/42 {name: "Alice", ...}
- Redis consumer: SET user:42 {name: "Alice", ...}
- Both converge. No dual-write in request path.

Crash after ES write, before offset commit:

- Restart, replay from last committed offset
- ES PUT same doc again → idempotent, no duplicate

Edge case: User searches immediately after profile update — ES may still show old name (async lag).
Fix: read-your-writes routing (query leader/DB for own data) or accept bounded staleness.

16. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
At-least-once + idempotent dedup via event ID	Retried sends must not double-notify	Notification system (APNS/FCM)
Stream-maintained materialized view	Precompute feed instead of fan-out read at request time	News feed system
Per-recipient inbox + consumer-offset sync	Multiple devices, independent resumable read positions	Chat system
Unbounded per-user pub/sub stream	Continuous location updates, not request/response	Nearby friends
Kafka stream feeding downstream consumers	Location/traffic updates reach ETA without polling	Google Maps
Partitions, offsets, consumer groups, exactly-once	The system <i>is</i> a message queue	Distributed message queue
Windowed aggregation + explicit aggregate point	Metrics roll up before storage, re-sliceable	Metrics monitoring & alerting
Watermarks, tumbling/sliding windows, offset-atomic-with-write	Late events guaranteed; double-counting = financial loss	Ad click event aggregation
CDC keeping derived cache in sync	Cache reflects DB without fragile dual-write	Hotel reservation (Redis inventory cache)
Async reindexing via Kafka	Search index consistent without dual-write in request path	Email service (Elasticsearch reindex)
Retry queue / dead-letter queue, multi-receiver pub-sub	Side effects on payment events retried safely, fanned out	Payment system
Event sourcing + CQRS, deterministic replay	Full audit trail, reconstruct any past state	Digital wallet

Event sourcing over append-only log	Matching engine state reconstructable for audit/replay	Stock exchange
-------------------------------------	--	----------------

17. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 3 Storage	Log-structured engines and compaction — same log mechanics.	"Kafka compaction is SSTable merge for messages."
Ch 5 Replication	Replication logs, LSNs, lag, read-your-writes — broker/consumer is leader/follower.	"Consumer offset = follower's log sequence number."
Ch 6 Partitioning	Topic partitioning is the same idea applied to logs.	"Partition count caps consumer parallelism."
Ch 7/9 Transactions	Atomic commit/2PC for exactly-once output; total order broadcast = log; fencing zombies.	"Exactly-once output needs atomic commit or idempotence."
Ch 8 Distributed Troubles	Unreliable clocks and networks are why event time is hard.	"Never window on processing time under lag."
Ch 10 Batch	Stream = batch with unbounded input; map-side joins → stream-table joins; exactly-once same discipline, incrementally.	"Batch and stream are the same math, different scheduling."

18. Key Takeaways

1. **Two broker families, two contracts:** AMQP/JMS = per-message ack, destructive, unordered under redelivery (task queues); log-based Kafka = partitioned, ordered, replayable (data integration, derived state).
2. **Database writes are an event stream;** CDC + log fixes dual-write by making one system leader and everything else followers.
3. **Log compaction** turns a topic into a durable full dataset — load-bearing trick for Kafka-as-storage and cheap derived view rebuilds.
4. **State is a cache of the log's latest values;** keep the log and re-derive anything, run old/new views side by side, recover from bad deploys.
5. **Distinguish event time from processing time;** pick window semantics deliberately; decide straggler policy upfront.
6. **All three join types** reduce to "maintain state from one side, probe from the other" — order-sensitive across streams.
7. **Exactly-once is a visible-effect guarantee** built from at-least-once + (atomic commit or idempotent writes keyed by offset), not a magic delivery mode.

19. Junior SWE Checkpoints

Q1: Kafka vs RabbitMQ — when do you pick each?

Kafka: ordered replay, derived state, data integration, fan-out to many consumers. **RabbitMQ:** task queue, async RPC, order unimportant, no replay needed. Different contracts — not interchangeable.

Q2: Why is dual-write dangerous?

No single leader orders writes across DB + index + cache. Concurrent clients race; partial failures desync. Silent divergence, no error raised. Fix: CDC — one system of record, derived stores follow the log.

Q3: What does log compaction enable for a new search index consumer?

Compacted topic holds latest value per key $\approx$ full DB copy. New consumer starts offset 0, scans to head, caught up — no snapshot coordination, no load on source DB.

Q4: Event time vs processing time — why does it matter?

Processing-time windows shift when processor lags or redeploys — backlog looks like traffic spike. Event-time windows are correct for analytics but stragglers arrive late; window never truly "complete."

Q5: How do you get exactly-once effect with at-least-once delivery?

Idempotent writes (set key=row, dedup by offset/event ID) or atomic commit of output + state + offset (2PC). Requires deterministic processing, identical replay order, fencing against zombie processors.

Q6: What's the difference between CDC and event sourcing?

CDC tails DB replication log — row-level changes, compaction applies (latest row per key). Event sourcing is app-level immutable domain events — later events don't supersede, compaction doesn't apply, snapshots for perf only.

Glossary

Term	Definition
Offset	Monotonic position per partition — replaces per-message acks
Consumer group	Cooperative consumers; each partition assigned to one member
Log compaction	Keep latest value per key; topic size $\approx$ DB size
CDC	Change Data Capture — tail replication log, publish as stream
Event sourcing	Append-only domain events; state by deterministic replay
Command vs event	Command = intent (may fail); event = immutable fact
Event time	When event occurred — correct for windowing
Straggler	Late-arriving event after window supposedly closed
Stream-stream join	Both streams windowed and indexed in local state
Stream-table join	Enrich stream from CDC-maintained local table replica
Exactly-once	Visible effect as-if-once — not magic, built from at-least-once + idempotence
CQRS	Command Query Responsibility Segregation — write shape $\neq$ read shape
Watermark	Estimate of how complete event-time progress is
Dead-letter queue	Failed messages routed for inspection/retry

